

Launcher de Unificação de Jogos e Comunidade

**Kaio Damasceno de Oliveira¹, Mateus Hideki de Figueiredo Tamura²,
Matheus Barbosa Silva³, Rafael Pires Goncalves⁴, Renan Trajano da Conceição⁵,
Thiago Akira Damasceno Taniguchi⁶, Ariel Paulino Gonzaga⁷**

¹Instituto Federal de Educação, Ciência e Tecnologia
Câmpus São Paulo (IFSP) São Paulo – SP – Brazil

{kaio.d, h.tamura, matheus.barbosa1, goncalves.pires, renan.trajano,
akira.taniguchi, barbara.gonzaga}@aluno.ifsp.edu.br

1. Introdução

A indústria de jogos digitais tem apresentado crescimento contínuo nas últimas décadas, impulsionado principalmente pela expansão das plataformas de distribuição digital. Esse cenário possibilitou o surgimento de serviços como *Steam*, *Epic Games* e outras lojas virtuais, que permitem aos usuários adquirir e gerenciar uma grande quantidade de títulos. Como consequência, as bibliotecas digitais tornaram-se cada vez maiores e mais diversificadas, aumentando a complexidade na organização e no acesso aos jogos.

Dados recentes reforçam a dimensão desse mercado. De acordo com a pesquisa *Global Gamer Study*, realizada com 76 mil jogadores, aproximadamente 43% dos usuários utilizam a plataforma PC como principal meio para jogos digitais (Newzoo, 2024). Além disso, plataformas como a *Steam* já registraram mais de 42 milhões de usuários jogando simultaneamente em março de 2026 (SteamDB, 2026), evidenciando a escala e a relevância desses ambientes digitais.

Nesse contexto, surgem os chamados *launchers de jogos*, aplicações responsáveis por instalar, organizar e executar títulos digitais, oferecendo ao usuário uma interface centralizada para gerenciamento de sua biblioteca. No entanto, como cada plataforma possui seu próprio *launcher*, o usuário frequentemente precisa utilizar múltiplas aplicações distintas para acessar todos os seus jogos, o que resulta em um ambiente fragmentado e pouco eficiente.

Essa fragmentação gera diversos problemas, como a descentralização das bibliotecas, a divisão de progresso entre plataformas e a necessidade de alternar entre diferentes aplicações para acessar conteúdos. Além disso, o grande volume de jogos disponíveis dificulta a descoberta de títulos relevantes, tornando a experiência do usuário menos eficiente. Para lidar com esse problema, sistemas de recomendação têm sido amplamente utilizados em plataformas digitais, sendo capazes de sugerir conteúdos com base no comportamento e nas preferências dos usuários (MARINHO et al., 2015).

Entretanto, nas plataformas atuais, os dados utilizados para recomendação são limitados ao ecossistema de cada serviço, o que reduz a precisão das sugestões, uma vez que não consideram o comportamento completo do usuário em diferentes ambientes. Esse cenário evidencia a necessidade de soluções que integrem informações de múltiplas fontes, permitindo uma análise mais abrangente e eficiente.

Diante disso, este trabalho propõe o desenvolvimento do *Salsilauncher*, um *launcher* de jogos capaz de integrar títulos provenientes de diferentes plataformas em uma única interface. O sistema permite o cadastro e execução de jogos de diversas origens, além de realizar o monitoramento automático do tempo de uso e integrar dados obtidos por meio de APIs externas. Com base nessas informações, é implementado um sistema de recomendação que utiliza o comportamento do usuário para sugerir novos jogos de interesse.

Adicionalmente, o sistema prevê a inclusão de funcionalidades sociais, possibilitando a interação entre usuários e o compartilhamento de atividades. Dessa forma, o *Salsilauncher* busca não apenas centralizar o acesso aos jogos, mas também melhorar a experiência do usuário por meio de organização, análise de dados e personalização.

1.1. Objetivos

1.1.1 Objetivo Geral

Desenvolver um *launcher* de jogos para *desktop* que permita organizar jogos locais, registrar automaticamente o tempo de execução e oferecer recomendações personalizadas baseadas no comportamento do usuário e na interação social entre jogadores.

1.1.2 Objetivos Específicos

- Implementar um sistema de biblioteca que permita adicionar e organizar jogos instalados localmente.
- Integrar a aplicação com a API da *Steam* para obtenção automática de informações sobre os jogos.
- Desenvolver um agente responsável por monitorar a execução dos jogos e registrar automaticamente o tempo de uso.
- Implementar um sistema de recomendação baseado em características dos jogos e no comportamento do usuário.
- Desenvolver funcionalidades sociais que permitam adicionar amigos e visualizar suas atividades.
- Implementar comunicação em tempo real entre usuários, incluindo chat e comunicação por voz.
- Criar um painel analítico para visualização de estatísticas de uso e preferências do jogador.

2. Referencial Teórico

2.1. Sistemas de Recomendação em *Python*

No contexto do *Salsilauncher*, um dos principais desafios é auxiliar o usuário na identificação de jogos relevantes dentro de sua própria biblioteca, reduzindo a necessidade de busca manual e melhorando a experiência de uso. Esse problema está diretamente relacionado ao grande volume de informações disponíveis e à ausência de mecanismos de curadoria personalizada, sendo amplamente tratado por sistemas de recomendação.

De acordo com Marinho et al. (MARINHO et al., 2015), sistemas de recomendação são responsáveis por sugerir itens relevantes com base em dados de interação dos usuários, utilizando informações como histórico de uso e preferências para prever interesses futuros. O estudo destaca que esses sistemas são aplicados em diversos contextos digitais, especialmente em ambientes com grande quantidade de conteúdo.

Entre as abordagens apresentadas, destaca-se a filtragem colaborativa, que utiliza dados de múltiplos usuários para identificar padrões de similaridade. Nesse modelo, assume-se que usuários com comportamentos semelhantes tendem a compartilhar interesses, permitindo a recomendação de novos itens com base nessas relações. Para isso, são utilizadas estruturas organizadas de dados e métricas matemáticas, como a similaridade entre vetores, que permitem quantificar o grau de proximidade entre elementos.

No *Salsilauncher*, essa abordagem é aplicada ao considerar dados como tempo de jogo e frequência de execução para representar a interação entre usuário e jogos. A partir dessas informações, o sistema pode identificar padrões de comportamento e calcular similaridades entre títulos, permitindo a geração de recomendações personalizadas. Dessa forma, o projeto se fundamenta diretamente nas técnicas apresentadas por Marinho et al., adaptando-as ao contexto específico de gerenciamento de bibliotecas de jogos digitais.

2.2. Desenvolvimento de Aplicações Web com *Frameworks Python*

O desenvolvimento do *Salsilauncher* envolve a integração de diferentes componentes responsáveis pela interface do usuário, processamento de dados e armazenamento de informações. Esse cenário exige uma arquitetura que permita comunicação eficiente entre as partes do sistema, garantindo desempenho e escalabilidade.

O estudo de análise comparativa de *frameworks web* (SCOTT; LEWIS, 2024) investiga a evolução do desenvolvimento de aplicações, destacando o papel de *frameworks* modernos na construção de sistemas mais eficientes e escaláveis. O trabalho evidencia que aplicações contemporâneas tendem a adotar arquiteturas baseadas em serviços, utilizando APIs para promover a separação entre *frontend* e *backend*.

Além disso, o artigo destaca o crescimento do uso de *frameworks Python*, ressaltando características como flexibilidade, produtividade e suporte a operações assíncronas. Essas propriedades tornam tais *frameworks* especialmente adequados para aplicações que necessitam lidar com múltiplas requisições e processamento contínuo de dados.

No *Salsilauncher*, esses conceitos são aplicados por meio da adoção de uma arquitetura baseada em API, na qual o *backend*, desenvolvido com *FastAPI*, é responsável por centralizar a lógica do sistema. Essa estrutura permite que o *frontend* interaja com o sistema de forma desacoplada, realizando operações como cadastro de jogos, recuperação de dados e solicitação de recomendações. Assim, o projeto segue diretamente as diretrizes apresentadas no estudo, garantindo organização e eficiência na comunicação entre seus componentes.

2.3. Algoritmos de Recomendação e Estruturação de Dados

A eficiência de sistemas de recomendação depende diretamente da forma como os dados são estruturados e dos algoritmos utilizados para processá-los. No *Salsilauncher*, esse desafio está presente na necessidade de representar as interações entre usuários e jogos de forma que permita identificar padrões e gerar recomendações relevantes.

O trabalho desenvolvido por Takahashi (TAKAHASHI, 2014) analisa diferentes algoritmos de recomendação e suas aplicações, destacando abordagens como filtragem colaborativa, filtragem baseada em conteúdo e sistemas híbridos. O estudo parte do cenário de crescimento do volume de dados digitais e da necessidade de extrair informações úteis a partir dessas interações.

A pesquisa demonstra que a escolha do algoritmo e a forma de organização dos dados são fatores determinantes para a qualidade das recomendações. Métodos baseados em conteúdo utilizam características dos itens e preferências do usuário, enquanto a filtragem colaborativa explora padrões de comportamento entre diferentes usuários. Além disso, o trabalho enfatiza a importância do tratamento adequado dos dados para garantir maior precisão nos resultados.

No nosso escopo, aplicados a ideia base na modelagem das interações entre usuários e jogos, utilizando dados como tempo de uso e frequência de execução. A partir dessa base, o sistema pode aplicar diferentes abordagens de recomendação para gerar sugestões personalizadas.

3. Materiais e Métodos

Esta seção apresenta os materiais e métodos utilizados no desenvolvimento do *Salsilauncher*. Inicialmente, são descritas as tecnologias e ferramentas empregadas na implementação do sistema. Em seguida, são detalhados os métodos adotados para a construção da aplicação, incluindo sua arquitetura, funcionamento e estratégias utilizadas para o desenvolvimento das funcionalidades propostas.

3.1. Materiais

3.1.1 Desenvolvimento *Front-end*

O *frontend* foi desenvolvido utilizando *React* e *TypeScript* (Meta Open Source, 2026; Microsoft, 2026b). A interface da aplicação é responsável pela visualização da biblioteca de jogos, interação com o usuário e envio de requisições ao *backend*.

A aplicação é empacotada como *software desktop* utilizando *Electron* (OpenJS Foundation, 2026), permitindo sua execução local e acesso direto a recursos do sistema operacional.

3.1.2 Desenvolvimento *Back-end*

O *backend* foi desenvolvido em *Python*, utilizando o *framework FastAPI* ([Python Software Foundation, 2026](#); [FastAPI, 2026](#)). Ele é responsável pelo processamento das requisições, gerenciamento dos dados e implementação das funcionalidades do sistema.

Para persistência dos dados, foi utilizado o *PostgreSQL* ([PostgreSQL Global Development Group, 2026](#)), onde são armazenadas informações como jogos cadastrados, tempo de uso, avaliações e dados dos usuários.

3.1.3 APIs Externas

O sistema prevê integração com a *Steam Web API* ([Valve Corporation, 2026](#)), utilizada para obtenção de informações sobre jogos, como nome, descrição, categorias e dados associados à biblioteca do usuário.. Essa integração permite que parte dos metadados seja preenchida automaticamente, reduzindo a necessidade de cadastro manual.

A arquitetura do sistema também considera a possibilidade de integração futura com outras plataformas que disponibilizem APIs públicas ou gratuitas. Dessa forma, a aplicação mantém uma estrutura extensível para ampliar o conjunto de fontes externas utilizadas na obtenção de dados sobre jogos.

3.1.4 Ferramentas de Desenvolvimento

Durante o desenvolvimento, foram utilizadas ferramentas como *Visual Studio Code* ([Microsoft, 2026c](#)), utilizado como ambiente de desenvolvimento, *GitHub* ([GitHub, 2026b](#)), empregado para versionamento de código, e *GitHub Projects* ([GitHub, 2026a](#)), utilizado para organização das tarefas. A comunicação e a organização da equipe foram realizadas por meio da plataforma *Discord* ([Discord, 2026](#)).

3.2. Métodos

3.2.1 Arquitetura do Sistema

O *Salsilauncher* foi concebido como uma aplicação capaz de centralizar o gerenciamento de jogos provenientes de diferentes fontes, permitindo ao usuário unificar sua biblioteca digital em um único ambiente. Para isso, o sistema foi projetado com base em uma arquitetura cliente-servidor, na qual o *frontend* e o *backend* operam de forma desacoplada, comunicando-se por meio de requisições HTTP.

3.2.2 Controle de Versão

O controle de versão do projeto é realizado com *Git* ([Git, 2026](#)), com o repositório hospedado no *GitHub* ([GitHub, 2026b](#)). Essa organização permite registrar o histórico de alterações do código, acompanhar as contribuições dos integrantes e facilitar a colaboração entre os membros da equipe.

Durante o desenvolvimento, os *commits* e *pull requests* seguem uma padronização interna, com o objetivo de manter o histórico do projeto organizado e facilitar a revisão das alterações realizadas.

3.2.3 Documentação

A documentação principal do projeto é produzida em $\text{\LaTeX}$, utilizando o *Overleaf* ([Overleaf, 2026](#)) como ambiente de edição colaborativa. As apresentações utilizadas nas entregas parciais são elaboradas com o *Google Slides* ([Google, 2026](#)). Essas ferramentas foram adotadas para organizar os artefatos acadêmicos do projeto e facilitar a edição colaborativa entre os integrantes da equipe.

3.2.4 Desenvolvimento Incremental

O desenvolvimento do projeto é realizado de forma incremental. Inicialmente, o foco está na construção da biblioteca de jogos, por ser a funcionalidade central do sistema. As demais funcionalidades, como recomendações, recursos sociais e análise de dados, são planejadas para etapas posteriores.

Essa abordagem permite que cada módulo seja desenvolvido, testado e aprimorado antes da inclusão de novas funcionalidades. Para auxiliar na organização das etapas de desenvolvimento, foi utilizado o *GitHub Projects*, no qual as tarefas foram separadas, categorizadas e acompanhadas pela equipe.

4. Requisitos

Nesta seção são apresentados os requisitos do *Salsilauncher*, divididos em requisitos funcionais e não funcionais. Os requisitos funcionais descrevem as funcionalidades que o sistema deve oferecer, enquanto os requisitos não funcionais definem restrições e características de qualidade do sistema.

4.1. Requisitos Funcionais

- RF01: O sistema deve permitir o cadastro manual de jogos.
- RF02: O sistema deve permitir a edição das informações de jogos cadastrados;
- RF03: O sistema deve permitir a remoção de jogos da biblioteca do usuário;
- RF04: O sistema deve exibir uma biblioteca contendo os jogos cadastrados pelo usuário;
- RF05: O sistema deve permitir a pesquisa de jogos por meio de filtros e palavras-chave;
- RF06: O sistema deve permitir a execução de jogos diretamente pela aplicação;
- RF07: O sistema deve registrar automaticamente o tempo de utilização dos jogos executados pelo *Salsilauncher*;
- RF08: O sistema deve armazenar o histórico de utilização dos jogos cadastrados;
- RF09: O sistema deve obter automaticamente metadados de jogos por meio de integrações com serviços externos;
- RF10: O sistema deve permitir a vinculação de contas de plataformas externas compatíveis;
- RF11: O sistema deve permitir a sincronização de bibliotecas de jogos provenientes de plataformas externas;
- RF12: O sistema deve permitir que usuários registrem avaliações para jogos cadastrados;
- RF13: O sistema deve gerar recomendações de jogos com base no histórico de utilização e avaliações dos usuários;
- RF14: O sistema deve disponibilizar informações analíticas relacionadas ao perfil de utilização do usuário;
- RF15: O sistema deve permitir o cadastro e autenticação de usuários;
- RF16: O sistema deve permitir o gerenciamento de amizades entre usuários;
- RF17: O sistema deve permitir a visualização de informações públicas de usuários adicionados como amigos;
- RF18: O sistema deve exportar e importar as informações da biblioteca do usuário, para que o mesmo a acesse de outros dispositivos;
- RF19: O sistema deve permitir a criação, edição e exclusão de coleções personalizadas de jogos na biblioteca do usuário;
- RF20: O sistema deve permitir a visualização e edição das informações de perfil do usuário;

4.2. Requisitos Não Funcionais

- RNF01: O sistema deve apresentar uma interface gráfica intuitiva e de fácil utilização;
- RNF02: O sistema deve ser executado como aplicação *desktop*;
- RNF03: O sistema deve ser compatível com o sistema operacional Windows;
- RNF04: O sistema deve possuir arquitetura que possibilite futura compatibilidade com sistemas Linux;
- RNF05: O sistema deve garantir a persistência dos dados dos usuários e jogos cadastrados;
- RNF06: O sistema deve garantir a integridade e consistência dos dados armazenados;
- RNF07: O sistema deve apresentar tempo de resposta adequado para operações de consulta e pesquisa;
- RNF08: O sistema deve proteger informações de autenticação dos usuários;
- RNF09: O sistema deve permitir integração com serviços externos de forma segura;
- RNF10: O sistema deve ser desenvolvido de forma modular, permitindo manutenção e expansão das funcionalidades;
- RNF11: O sistema deve suportar a adição de novas integrações com plataformas de jogos sem necessidade de reformulação completa da arquitetura;

5. Modelagem

Esta seção apresenta a modelagem inicial do *Salsilauncher*, com foco nos casos de uso identificados para a aplicação. A modelagem foi dividida em três grupos principais: biblioteca de jogos, integração e recomendação, e funcionalidades sociais com *dashboard* analítico.

5.1. Casos de Uso

Os casos de uso representam as principais interações entre os usuários e o sistema. Eles descrevem as funcionalidades previstas para o *Salsilauncher*, indicando quais ações podem ser executadas pelo usuário e quais integrações externas são utilizadas pela aplicação.

5.1.1 Biblioteca

O diagrama a seguir apresenta os casos de uso relacionados ao módulo de biblioteca de jogos. Esse módulo concentra as funcionalidades centrais do *Salsilauncher*, como cadastro, edição, remoção, pesquisa, execução de jogos, visualização de tempo de uso, gerenciamento de coleções e importação de dados de perfil.

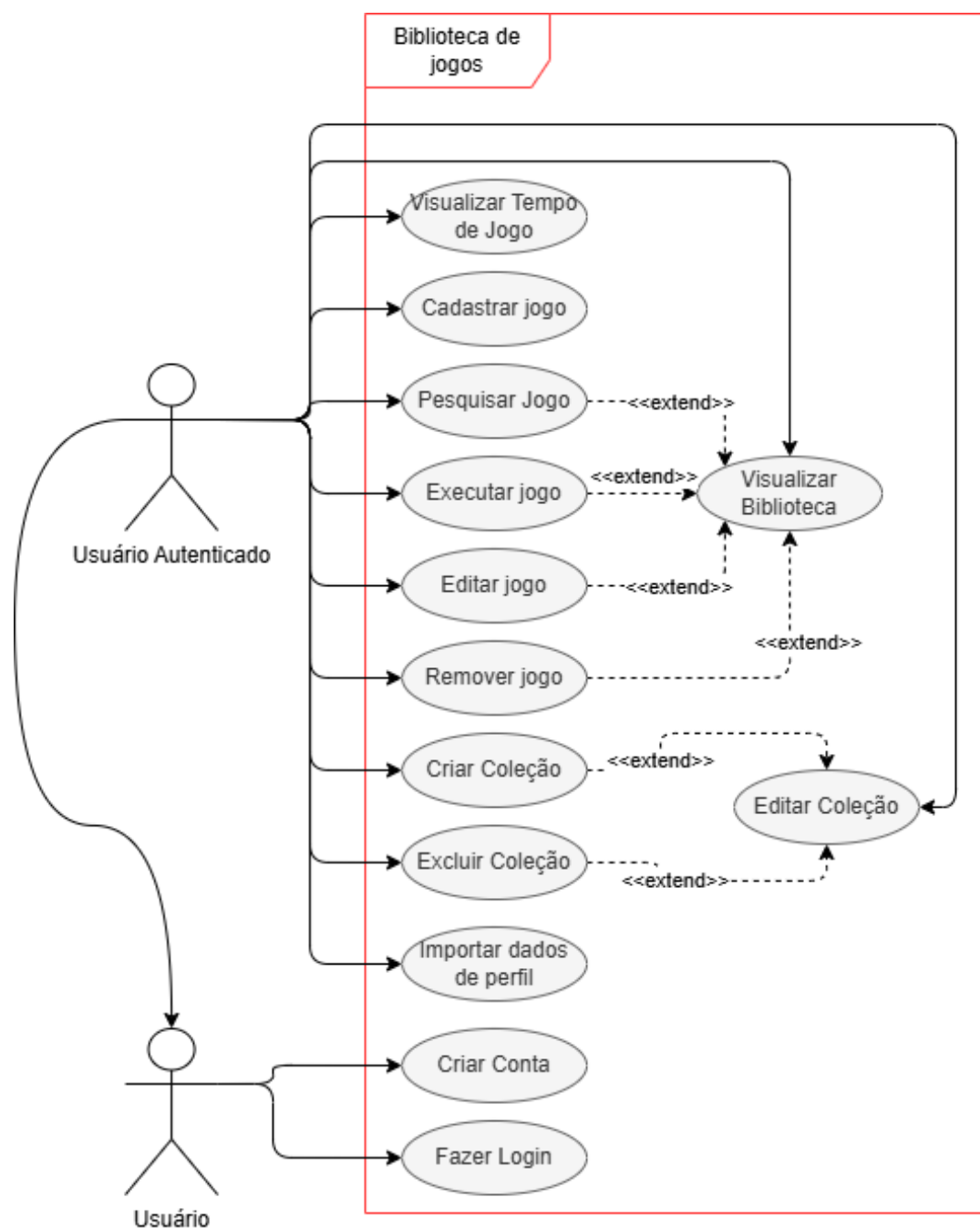


Figura 1 – Diagrama de casos de uso da Biblioteca de Jogos

5.1.2 Integração e Recomendação

O diagrama a seguir apresenta os casos de uso relacionados às funcionalidades de integração com serviços externos e ao sistema de recomendação. Nesse grupo estão incluídas a importação da biblioteca da *Steam*, a avaliação de jogos e a visualização de recomendações personalizadas.

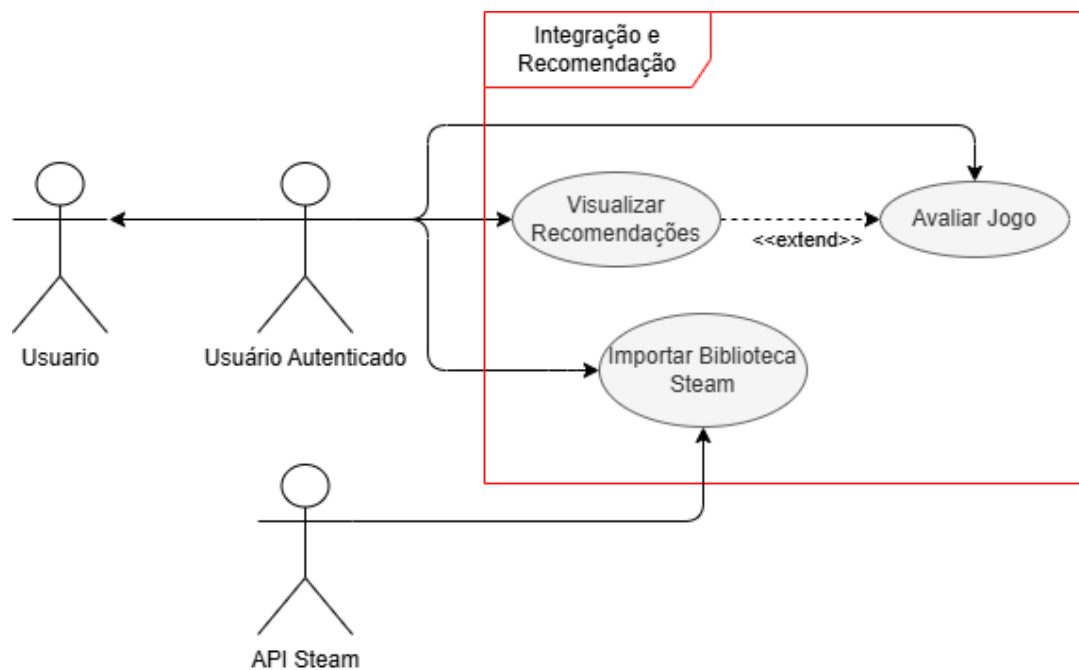


Figura 2 – Diagrama de casos de uso de Integração e Recomendação

5.1.3 Social e *Dashboard*

O diagrama a seguir apresenta os casos de uso relacionados às funcionalidades sociais e ao *dashboard* analítico. Essas funcionalidades permitem que o usuário visualize perfis, gerencie amizades, consulte atividades de amigos e acesse informações analíticas sobre seu uso da aplicação.

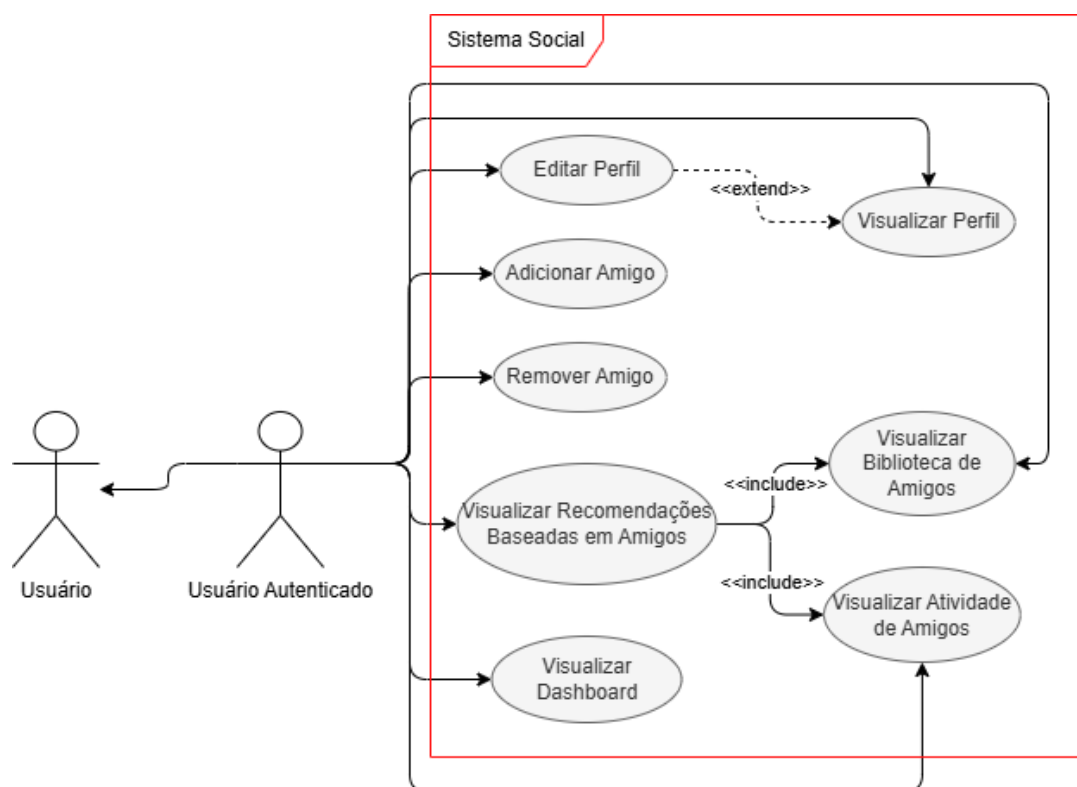


Figura 3 – Diagrama de casos de uso do sistema Social e *Dashboard* Analítico

As especificações completas dos casos de uso foram organizadas no Apêndice A. Como exemplo, a tabela a seguir apresenta a especificação do caso de uso de cadastro de jogo, uma das funcionalidades centrais da biblioteca.

Tabela 1 – Caso de Uso UC-03 - Cadastrar Jogo

Objetivo	Permitir que o usuário adicione manualmente um jogo à sua biblioteca.
RF/RNF Relacionados	RF01, RF09, RNF05
Atores	Usuário
Prioridade	Alta
Pré-condições	O usuário deve estar autenticado.
Frequência de uso	Alta
Criticalidade	Alta
Condição de Entrada	Usuário seleciona a opção de cadastrar jogo.
Fluxo Principal	1. O usuário acessa a tela de cadastro de jogo. 2. O sistema solicita as informações do jogo. 3. O usuário informa os dados necessários, incluindo o caminho do executável. 4. O sistema valida as informações fornecidas. 5. O sistema busca metadados automaticamente, quando possível. 6. O sistema adiciona o jogo à biblioteca do usuário.
Fluxo Alternativo	Caso o caminho do executável seja inválido ou inexistente, o sistema exibe uma mensagem de erro e cancela o cadastro.
Pós-condições	O jogo é armazenado e passa a ser exibido na biblioteca do usuário.
Regras de Negócio	O caminho do executável deve ser válido e acessível pelo sistema.

6. Protótipo

O protótipo atual do *Salsilauncher* foi desenvolvido com o objetivo de representar visualmente a estrutura inicial da aplicação e validar a organização das principais telas previstas para o sistema. Nesta etapa, o protótipo contempla principalmente as funcionalidades relacionadas à autenticação de usuários, navegação pela interface, visualização da biblioteca e cadastro de jogos.

As telas foram elaboradas com foco em uma interface escura, utilizando elementos visuais em vermelho para destacar ações principais e áreas selecionadas. Essa identidade visual busca manter consistência entre as telas e facilitar a identificação das funcionalidades disponíveis ao usuário.

O protótipo inclui telas de login, criação de conta, página inicial da biblioteca, menu lateral, área de coleções, cadastro manual de jogos e escaneamento de pastas. Essas telas servem como referência para o desenvolvimento do *frontend* e para a validação inicial da experiência de uso.

As imagens completas do protótipo estão disponíveis no Apêndice 2.

7. Plano de Testes

O plano de testes tem como objetivo verificar se as funcionalidades do *Salsilauncher* atendem aos requisitos especificados para o sistema. Os testes foram definidos com base nos casos de uso e nas funcionalidades previstas para a aplicação, contemplando tanto requisitos funcionais quanto requisitos não funcionais.

Os testes do *Salsilauncher* serão realizados de forma incremental ao longo do desenvolvimento. A validação das funcionalidades será conduzida por meio de testes automatizados e testes funcionais.

Os testes automatizados serão divididos em duas frentes principais:

- **Backend (FastAPI e SQLAlchemy):** utilização do *framework* *Pytest* (pytest, 2026) para validação de *endpoints*, regras de negócio e integração do sistema. Serão empregadas técnicas de *mocking* para simular APIs externas e operações de banco de dados durante a esteira de CI/CD.
- **Frontend (React + TypeScript):** utilização de *Jest* e *React Testing Library* (Jest, 2026; Testing Library, 2026) para testes unitários e de componentes, além de *Playwright* ou *Cypress* (Microsoft, 2026a; Cypress, 2026) para testes *End-to-End*.

Os testes serão executados continuamente durante o desenvolvimento, garantindo a detecção precoce de falhas e a prevenção de regressões. As tabelas completas dos casos de teste estão disponíveis no Apêndice C.

A tabela a seguir apresenta um exemplo de caso de teste voltado ao cadastro de jogos, funcionalidade essencial para o funcionamento da biblioteca do *Salsilauncher*.

Tabela 2 – Caso de teste CT-01

Identificador	CT-01
Funcionalidade	Cadastro de jogo
Objetivo	Verificar se um jogo pode ser adicionado à biblioteca.
Pré-condição	Nenhuma.
Entrada	Nome do jogo e caminho válido do executável.
Resultado esperado	O jogo deve ser salvo e exibido na biblioteca do usuário.
Exceção (Caminho Triste)	Se o caminho do executável for inválido ou inexistente, o sistema deve barrar o cadastro e exibir uma mensagem de erro ("Caminho do arquivo não encontrado").

Referências

- Cypress. *Cypress Documentation*. 2026. <<https://docs.cypress.io/>>. Acesso em: 09 junho 2026. Citado na página 10.
- Discord. *Discord Support*. 2026. <<https://support.discord.com/>>. Acesso em: 03 junho 2026. Citado na página 4.
- FastAPI. *FastAPI Documentation*. 2026. Acesso em: 13 maio 2026. Disponível em: <<https://fastapi.tiangolo.com/>>. Citado na página 4.
- Git. *Git Reference*. 2026. Acesso em: 22 maio 2026. Disponível em: <<https://git-scm.com/docs>>. Citado na página 4.
- GitHub. *About Projects*. 2026. Acesso em: 22 maio 2026. Disponível em: <<https://docs.github.com/issues/planning-and-tracking-with-projects/learning-about-projects/about-projects>>. Citado na página 4.
- GitHub. *GitHub Docs*. 2026. <<https://docs.github.com/>>. Acesso em: 29 maio 2026. Citado na página 4.
- Google. *Ajuda do Apresentações Google*. 2026. <<https://support.google.com/docs/topic/9054003?hl=pt-BR>>. Acesso em: 03 junho 2026. Citado na página 4.
- Jest. *Jest Documentation*. 2026. <<https://jestjs.io/docs/getting-started>>. Acesso em: 06 junho 2026. Citado na página 10.
- MARINHO, L. H. et al. A recommender system in python. 2015. Acesso em: 4 abril 2026. Disponível em: <<https://www.academia.edu/download/72929413/331-1.pdf>>. Citado nas páginas 1 e 2.
- Meta Open Source. *React Documentation*. 2026. Acesso em: 13 maio 2026. Disponível em: <<https://react.dev/>>. Citado na página 3.
- Microsoft. *Playwright Documentation*. 2026. <<https://playwright.dev/>>. Acesso em: 09 junho 2026. Citado na página 10.
- Microsoft. *TypeScript Documentation*. 2026. Acesso em: 15 maio 2026. Disponível em: <<https://www.typescriptlang.org/docs/>>. Citado na página 3.
- Microsoft. *Visual Studio Code Documentation*. 2026. <<https://code.visualstudio.com/docs>>. Acesso em: 29 maio 2026. Citado na página 4.
- Newzoo. *Global Gamer Study 2024*. 2024. <<https://newzoo.com/global-gamer-study>>. Pesquisas globais sobre o comportamento gamer. Acesso em: 02 abril 2026. Citado na página 1.
- OpenJS Foundation. *Electron Documentation*. 2026. Acesso em: 15 maio 2026. Disponível em: <<https://www.electronjs.org/docs/latest>>. Citado na página 3.
- Overleaf. *Overleaf Documentation*. 2026. <<https://docs.overleaf.com/>>. Acesso em: 03 junho 2026. Citado na página 4.
- PostgreSQL Global Development Group. *PostgreSQL Documentation*. 2026. Acesso em: 20 maio 2026. Disponível em: <<https://www.postgresql.org/docs/>>. Citado na página 4.
- pytest. *pytest documentation*. 2026. <<https://docs.pytest.org/en/stable/>>. Acesso em: 06 junho 2026. Citado na página 10.
- Python Software Foundation. *Python Documentation*. 2026. <<https://docs.python.org/3/>>. Acesso em: 27 maio 2026. Citado na página 4.
- SCOTT, C.; LEWIS, B. Advancements in web application development: An analytical review of ruby on rails, python frameworks and cloud-centric solutions. 2024. Acesso em: 4 abril 2026. Disponível em: <<https://www.researchgate.net/publication/394759430>>. Citado na página 3.

SteamDB. *Steam Concurrent Players Statistics*. 2026. <<https://steamdb.info>>. Banco de informações sobre a plataforma Steam. Acesso em: 02 abril 2026. Citado na página 1.

TAKAHASHI, M. M. *Algoritmos de Recomendação*. 2014. <<https://bccdev.ime.usp.br/tccs/2014/marcost/index.html>>. Trabalho de Conclusão de Curso, Instituto de Matemática e Estatística da Universidade de São Paulo. Acesso em: 4 abril 2026. Citado na página 3.

Testing Library. *React Testing Library*. 2026. <<https://testing-library.com/docs/react-testing-library/intro/>>. Acesso em: 06 junho 2026. Citado na página 10.

Valve Corporation. *Steam Web API Documentation*. 2026. Acesso em: 20 maio 2026. Disponível em: <https://developer.valvesoftware.com/wiki/Steam_Web_API>. Citado na página 4.

APÊNDICE A – Tabelas de Casos de Uso

1. Tabelas de Casos de Uso

Este apêndice apresenta as tabelas completas de especificação dos casos de uso do *Salsilauncher*. Cada tabela descreve o objetivo do caso de uso, requisitos relacionados, atores envolvidos, prioridade, pré-condições, frequência de uso, criticidade, condição de entrada, fluxo principal, fluxo alternativo, pós-condições e regras de negócio.

1.1. Biblioteca

A tabela a seguir apresenta a especificação do caso de uso UC-01 - Criar Conta, responsável por permitir o cadastro inicial de novos usuários na plataforma.

Tabela 3 – Caso de Uso UC-01 - Criar Conta

Objetivo	Permitir que um novo usuário realize seu cadastro na plataforma.
RF/RNF Relacionados	RF15, RNF08
Atores	Usuário
Prioridade	Alta
Pré-condições	Nenhuma.
Frequência de uso	Baixa
Criticidade	Alta
Condição de Entrada	Usuário seleciona a opção de cadastro.
Fluxo Principal	1. O usuário acessa a tela de cadastro. 2. O sistema solicita o preenchimento dos seguintes campos: • E-mail • Senha 3. O usuário informa os dados solicitados. 4. O sistema valida as informações. 5. O sistema cria a conta do usuário.
Fluxo Alternativo	Caso já exista uma conta associada ao e-mail informado, o sistema exibe uma mensagem de erro e cancela a operação.
Pós-condições	Uma nova conta é criada e armazenada no sistema.
Regras de Negócio	Cada usuário deve possuir um identificador único.

A tabela a seguir apresenta a especificação do caso de uso UC-02 - Fazer Login, responsável por permitir que usuários cadastrados acessem suas contas no sistema.

Tabela 4 – Caso de Uso UC-02 - Fazer Login

Objetivo	Permitir que um usuário cadastrado acesse sua conta na plataforma.
RF/RNF Relacionados	RF15, RNF08
Atores	Usuário
Prioridade	Alta
Pré-condições	O usuário deve possuir uma conta cadastrada.
Frequência de uso	Alta
Criticidade	Alta
Condição de Entrada	Usuário seleciona a opção de login.

Fluxo Principal	1. O usuário acessa a tela de login. 2. O sistema solicita o preenchimento dos seguintes campos: • E-mail • Senha 3. O usuário informa suas credenciais. 4. O sistema valida as credenciais informadas. 5. O sistema permite o acesso à conta do usuário.
Fluxo Alternativo	Caso as credenciais estejam incorretas, o sistema exibe uma mensagem de erro e impede o acesso.
Pós-condições	O usuário passa a ter uma sessão autenticada no sistema.
Regras de Negócio	Somente usuários cadastrados podem acessar o sistema.

A tabela a seguir apresenta a especificação do caso de uso UC-03 - Cadastrar Jogo, relacionado ao cadastro manual de jogos na biblioteca do usuário.

Tabela 5 – Caso de Uso UC-03 - Cadastrar Jogo

Objetivo	Permitir que o usuário adicione manualmente um jogo à sua biblioteca.
RF/RNF Relacionados	RF01, RF09, RNF05
Atores	Usuário
Prioridade	Alta
Pré-condições	O usuário deve estar autenticado.
Frequência de uso	Alta
Criticalidade	Alta
Condição de Entrada	Usuário seleciona a opção de cadastrar jogo.
Fluxo Principal	1. O usuário acessa a tela de cadastro de jogo. 2. O sistema solicita as informações do jogo. 3. O usuário informa os dados necessários, incluindo o caminho do executável. 4. O sistema valida as informações fornecidas. 5. O sistema busca metadados automaticamente, quando possível. 6. O sistema adiciona o jogo à biblioteca do usuário.
Fluxo Alternativo	Caso o caminho do executável seja inválido ou inexistente, o sistema exibe uma mensagem de erro e cancela o cadastro.
Pós-condições	O jogo é armazenado e passa a ser exibido na biblioteca do usuário.
Regras de Negócio	O caminho do executável deve ser válido e acessível pelo sistema.

A tabela a seguir apresenta a especificação do caso de uso UC-04 - Editar Jogo, responsável por permitir a alteração das informações de jogos previamente cadastrados.

Tabela 6 – Caso de Uso UC-04 - Editar Jogo

Objetivo	Permitir que o usuário altere informações de um jogo cadastrado.
RF/RNF Relacionados	RF02, RNF06
Atores	Usuário
Prioridade	Média
Pré-condições	O jogo deve estar cadastrado na biblioteca do usuário.
Frequência de uso	Média
Criticalidade	Média
Condição de Entrada	Usuário seleciona a opção de editar jogo.

Fluxo Principal	1. O usuário seleciona um jogo cadastrado. 2. O usuário acessa a opção de edição. 3. O sistema exibe as informações atuais do jogo. 4. O usuário altera os dados desejados. 5. O sistema valida as alterações. 6. O sistema atualiza as informações do jogo.
Fluxo Alternativo	Caso os dados informados sejam inválidos, o sistema exibe uma mensagem de erro e mantém as informações anteriores.
Pós-condições	As informações do jogo são atualizadas no sistema.
Regras de Negócio	Somente jogos pertencentes ao usuário podem ser editados.

A tabela a seguir apresenta a especificação do caso de uso UC-05 - Remover Jogo, relacionado à remoção de jogos da biblioteca do usuário.

Tabela 7 – Caso de Uso UC-05 - Remover Jogo

Objetivo	Permitir que o usuário remova um jogo de sua biblioteca.
RF/RNF Relacionados	RF03, RNF06
Atores	Usuário
Prioridade	Média
Pré-condições	O jogo deve estar cadastrado na biblioteca do usuário.
Frequência de uso	Baixa
Criticalidade	Média
Condição de Entrada	Usuário seleciona a opção de remover jogo.
Fluxo Principal	1. O usuário seleciona um jogo cadastrado. 2. O usuário escolhe a opção de remoção. 3. O sistema solicita confirmação da ação. 4. O usuário confirma a remoção. 5. O sistema remove o jogo da biblioteca do usuário.
Fluxo Alternativo	Caso o usuário cancele a confirmação, nenhuma alteração é realizada.
Pós-condições	O jogo deixa de ser exibido na biblioteca do usuário.
Regras de Negócio	A remoção do jogo da biblioteca não deve excluir os arquivos físicos do dispositivo.

A tabela a seguir apresenta a especificação do caso de uso UC-06 - Pesquisar Jogo, responsável por permitir a localização de jogos cadastrados por meio de filtros e palavras-chave.

Tabela 8 – Caso de Uso UC-06 - Pesquisar Jogo

Objetivo	Permitir que o usuário localize jogos cadastrados em sua biblioteca.
RF/RNF Relacionados	RF05, RNF07
Atores	Usuário
Prioridade	Alta
Pré-condições	O usuário deve estar autenticado.
Frequência de uso	Alta
Criticalidade	Média
Condição de Entrada	Usuário utiliza a barra de pesquisa ou filtros disponíveis.
Fluxo Principal	1. O usuário acessa a biblioteca. 2. O usuário informa uma palavra-chave ou seleciona filtros de pesquisa. 3. O sistema processa os critérios informados. 4. O sistema exibe os jogos correspondentes à pesquisa.
Fluxo Alternativo	Caso nenhum jogo corresponda aos critérios informados, o sistema exibe uma mensagem indicando que nenhum resultado foi encontrado.
Pós-condições	A biblioteca é exibida de acordo com os critérios de pesquisa aplicados.

Regras de Negócio	A pesquisa deve considerar apenas os jogos pertencentes ao usuário.
-------------------	---

A tabela a seguir apresenta a especificação do caso de uso UC-07 - Visualizar Biblioteca, relacionado à exibição dos jogos cadastrados pelo usuário.

Tabela 9 – Caso de Uso UC-07 - Visualizar Biblioteca

Objetivo	Permitir que o usuário visualize todos os jogos cadastrados em sua biblioteca.
RF/RNF Relacionados	RF04, RNF01
Atores	Usuário
Prioridade	Alta
Pré-condições	O usuário deve estar autenticado.
Frequência de uso	Alta
Criticalidade	Alta
Condição de Entrada	Usuário acessa a tela principal da biblioteca.
Fluxo Principal	1. O usuário acessa a tela da biblioteca. 2. O sistema recupera os jogos cadastrados pelo usuário. 3. O sistema organiza os jogos para exibição. 4. O sistema apresenta a biblioteca ao usuário.
Fluxo Alternativo	Caso não existam jogos cadastrados, o sistema exibe a biblioteca vazia.
Pós-condições	A biblioteca do usuário é apresentada na interface.
Regras de Negócio	Cada usuário deve visualizar apenas sua própria biblioteca.

A tabela a seguir apresenta a especificação do caso de uso UC-08 - Executar Jogo, responsável por permitir a inicialização de jogos diretamente pelo *Salsilauncher*.

Tabela 10 – Caso de Uso UC-08 - Executar Jogo

Objetivo	Permitir que o usuário execute jogos diretamente pelo <i>Salsilauncher</i> .
RF/RNF Relacionados	RF06, RF07, RNF02, RNF03
Atores	Usuário
Prioridade	Alta
Pré-condições	O jogo deve estar cadastrado e possuir um executável válido.
Frequência de uso	Alta
Criticalidade	Alta
Condição de Entrada	Usuário seleciona a opção de executar jogo.
Fluxo Principal	1. O usuário seleciona um jogo cadastrado. 2. O usuário escolhe a opção de executar jogo. 3. O sistema verifica o caminho do executável associado. 4. O sistema inicia o jogo. 5. O sistema registra automaticamente o tempo de utilização durante a execução.
Fluxo Alternativo	Caso o executável não seja encontrado, o sistema exibe uma mensagem de erro e cancela a execução.
Pós-condições	O jogo é iniciado e seu tempo de uso passa a ser registrado.
Regras de Negócio	Apenas jogos com caminho de execução válido podem ser iniciados.

A tabela a seguir apresenta a especificação do caso de uso UC-09 - Visualizar Tempo de Jogo, relacionado à consulta do tempo de utilização registrado para os jogos cadastrados.

Tabela 11 – Caso de Uso UC-09 - Visualizar Tempo de Jogo

Objetivo	Permitir que o usuário consulte o tempo de utilização registrado para seus jogos.
RF/RNF Relacionados	RF07, RF08, RNF05
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve estar autenticado e possuir jogos cadastrados.
Frequência de uso	Média
Criticalidade	Média
Condição de Entrada	Usuário acessa as informações de tempo de jogo.
Fluxo Principal	1. O usuário acessa a área de tempo de jogo. 2. O sistema recupera os registros de utilização armazenados. 3. O sistema calcula o tempo total de uso dos jogos. 4. O sistema apresenta o tempo de jogo ao usuário.
Fluxo Alternativo	Caso não existam registros de utilização, o sistema exibe tempo igual a zero.
Pós-condições	O usuário visualiza o tempo de utilização registrado para seus jogos.
Regras de Negócio	O tempo de jogo deve ser calculado com base nos registros armazenados pelo sistema.

A tabela a seguir apresenta a especificação do caso de uso UC-10 - Criar Coleção, responsável por permitir a organização de jogos em coleções personalizadas.

Tabela 12 – Caso de Uso UC-10 - Criar Coleção

Objetivo	Permitir que o usuário organize jogos em coleções personalizadas.
RF/RNF Relacionados	RF19, RNF01, RNF05
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve estar autenticado.
Frequência de uso	Média
Criticalidade	Baixa
Condição de Entrada	Usuário seleciona a opção de criar coleção.
Fluxo Principal	1. O usuário acessa a área de coleções. 2. O usuário seleciona a opção de criar uma nova coleção. 3. O sistema solicita o nome da coleção. 4. O usuário informa o nome desejado. 5. O sistema cria a coleção e a associa ao usuário.
Fluxo Alternativo	Caso o nome informado seja inválido ou vazio, o sistema exibe uma mensagem de erro e cancela a criação.
Pós-condições	Uma nova coleção é criada e fica disponível para organização dos jogos.
Regras de Negócio	Cada coleção deve possuir um nome válido e estar associada a um usuário.

A tabela a seguir apresenta a especificação do caso de uso UC-11 - Editar Coleção, relacionado à alteração das informações de coleções previamente criadas.

Tabela 13 – Caso de Uso UC-11 - Editar Coleção

Objetivo	Permitir que o usuário altere informações de uma coleção existente.
RF/RNF Relacionados	RF19, RNF06
Atores	Usuário
Prioridade	Baixa
Pré-condições	O usuário deve possuir uma coleção previamente criada.
Frequência de uso	Baixa

Criticalidade	Baixa
Condição de Entrada	Usuário seleciona a opção de editar coleção.
Fluxo Principal	1. O usuário acessa suas coleções. 2. O usuário seleciona uma coleção existente. 3. O usuário escolhe a opção de edição. 4. O sistema exibe as informações atuais da coleção. 5. O usuário altera os dados desejados. 6. O sistema salva as alterações.
Fluxo Alternativo	Caso os dados informados sejam inválidos, o sistema exibe uma mensagem de erro e mantém as informações anteriores.
Pós-condições	A coleção é atualizada no sistema.
Regras de Negócio	Somente o proprietário da coleção pode editá-la.

A tabela a seguir apresenta a especificação do caso de uso UC-12 - Excluir Coleção, responsável por permitir a remoção de coleções sem excluir os jogos associados a elas.

Tabela 14 – Caso de Uso UC-12 - Excluir Coleção

Objetivo	Permitir que o usuário remova uma coleção existente.
RF/RNF Relacionados	RF19, RNF06
Atores	Usuário
Prioridade	Baixa
Pré-condições	O usuário deve possuir uma coleção previamente criada.
Frequência de uso	Baixa
Criticalidade	Baixa
Condição de Entrada	Usuário seleciona a opção de excluir coleção.
Fluxo Principal	1. O usuário acessa suas coleções. 2. O usuário seleciona uma coleção existente. 3. O usuário escolhe a opção de exclusão. 4. O sistema solicita confirmação da ação. 5. O usuário confirma a exclusão. 6. O sistema remove a coleção.
Fluxo Alternativo	Caso o usuário cancele a confirmação, nenhuma alteração é realizada.
Pós-condições	A coleção deixa de estar disponível para o usuário.
Regras de Negócio	A exclusão da coleção não deve remover os jogos da biblioteca do usuário.

A tabela a seguir apresenta a especificação do caso de uso UC-13 - Importar Dados de Perfil, relacionado à recuperação dos dados do usuário em outro dispositivo.

Tabela 15 – Caso de Uso UC-13 - Importar Dados de Perfil

Objetivo	Permitir que o usuário importe seus dados de perfil em outro dispositivo.
RF/RNF Relacionados	RF18, RNF05, RNF06, RNF08
Atores	Usuário
Prioridade	Alta
Pré-condições	O usuário deve possuir uma conta cadastrada com dados armazenados no servidor.
Frequência de uso	Baixa
Criticalidade	Alta
Condição de Entrada	Usuário realiza login em um novo dispositivo ou solicita a importação dos dados.

Fluxo Principal	1. O usuário autentica sua conta no dispositivo. 2. O sistema verifica a existência de dados armazenados no servidor. 3. O sistema recupera biblioteca, tempo de jogo, avaliações e demais informações do perfil. 4. O sistema importa os dados para o dispositivo atual. 5. O sistema disponibiliza as informações ao usuário.
Fluxo Alternativo	Caso a importação falhe, o sistema exibe uma mensagem de erro e mantém os dados locais inalterados.
Pós-condições	Os dados do perfil ficam disponíveis no dispositivo atual.
Regras de Negócio	Somente o proprietário da conta pode importar os dados associados ao seu perfil.

1.2. Integração e Recomendação

A tabela a seguir apresenta a especificação do caso de uso UC-14 - Avaliar Jogo, responsável por permitir que o usuário registre avaliações para jogos cadastrados.

Tabela 16 – Caso de Uso UC-14 - Avaliar Jogo

Objetivo	Permitir que o usuário registre uma avaliação para jogos cadastrados.
RF/RNF Relacionados	RF12, RNF05, RNF06
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve estar autenticado e o jogo deve estar cadastrado na biblioteca.
Frequência de uso	Média
Criticalidade	Média
Condição de Entrada	Usuário seleciona a opção de avaliar um jogo.
Fluxo Principal	1. O usuário seleciona um jogo cadastrado. 2. O usuário acessa a opção de avaliação. 3. O sistema solicita a nota ou opinião do usuário. 4. O usuário informa sua avaliação. 5. O sistema valida e armazena a avaliação.
Fluxo Alternativo	Caso a avaliação informada seja inválida, o sistema exibe uma mensagem de erro e solicita correção.
Pós-condições	A avaliação do jogo é registrada no sistema.
Regras de Negócio	Cada usuário deve possuir apenas uma avaliação ativa por jogo, podendo editá-la posteriormente.

A tabela a seguir apresenta a especificação do caso de uso UC-15 - Visualizar Recomendações, relacionado à exibição de sugestões de jogos com base no histórico de uso e avaliações do usuário.

Tabela 17 – Caso de Uso UC-15 - Visualizar Recomendações

Objetivo	Permitir que o usuário visualize recomendações de jogos com base em seu histórico de uso e avaliações.
RF/RNF Relacionados	RF13, RF14, RNF01, RNF07
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve estar autenticado e possuir dados suficientes de uso ou avaliação.
Frequência de uso	Média
Criticalidade	Média
Condição de Entrada	Usuário acessa a área de recomendações.

Fluxo Principal	1. O usuário acessa a tela de recomendações. 2. O sistema analisa o histórico de uso, avaliações e dados da biblioteca. 3. O sistema gera uma lista de jogos recomendados. 4. O sistema exibe as recomendações ao usuário.
Fluxo Alternativo	Caso não existam dados suficientes, o sistema exibe recomendações genéricas ou informa que ainda não há recomendações disponíveis.
Pós-condições	O usuário visualiza uma lista de jogos recomendados.
Regras de Negócio	As recomendações devem considerar o histórico de utilização e as avaliações registradas pelo usuário.

A tabela a seguir apresenta a especificação do caso de uso UC-16 - Importar Biblioteca *Steam*, responsável por permitir a importação de jogos provenientes de uma biblioteca *Steam* vinculada.

Tabela 18 – Caso de Uso UC-16 - Importar Biblioteca *Steam*

Objetivo	Permitir que o usuário importe jogos de sua biblioteca <i>Steam</i> para o <i>Salsilauncher</i> .
RF/RNF Relacionados	RF09, RF10, RF11, RNF09, RNF11
Atores	Usuário, <i>Steam</i> API
Prioridade	Alta
Pré-condições	O usuário deve estar autenticado e possuir uma conta <i>Steam</i> compatível para vinculação.
Frequência de uso	Baixa
Criticalidade	Alta
Condição de Entrada	Usuário seleciona a opção de importar biblioteca <i>Steam</i> .
Fluxo Principal	1. O usuário acessa a área de integrações. 2. O usuário seleciona a opção de importar biblioteca <i>Steam</i>. 3. O sistema solicita a vinculação ou identificação da conta <i>Steam</i>. 4. O sistema consulta os dados disponíveis por meio da <i>Steam</i> API. 5. O sistema importa os jogos encontrados. 6. O sistema adiciona os jogos importados à biblioteca do usuário.
Fluxo Alternativo	Caso a conta <i>Steam</i> não possa ser acessada ou a integração falhe, o sistema exibe uma mensagem de erro e cancela a importação.
Pós-condições	Os jogos importados da <i>Steam</i> passam a ser exibidos na biblioteca do usuário.
Regras de Negócio	Apenas dados autorizados e disponíveis pela integração externa devem ser importados.

1.3. Social e Dashboard

A tabela a seguir apresenta a especificação do caso de uso UC-17 - Visualizar Recomendações Baseadas em Amigos, relacionado à geração de sugestões considerando informações públicas de usuários adicionados como amigos.

Tabela 19 – Caso de Uso UC-17 - Visualizar Recomendações Baseadas em Amigos

Objetivo	Permitir que o usuário visualize recomendações de jogos com base nas bibliotecas e atividades de seus amigos.
RF/RNF Relacionados	RF13, RF16, RF17, RNF01, RNF07
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve estar autenticado e possuir amigos adicionados.
Frequência de uso	Média

Criticalidade	Média
Condição de Entrada	Usuário acessa a área de recomendações sociais.
Fluxo Principal	1. O usuário acessa a área de recomendações baseadas em amigos. 2. O sistema consulta informações públicas dos amigos adicionados. 3. O sistema identifica jogos em comum, jogos populares entre amigos e possíveis interesses do usuário. 4. O sistema gera recomendações sociais. 5. O sistema exibe as recomendações ao usuário.
Fluxo Alternativo	Caso o usuário não possua amigos adicionados ou não existam dados suficientes, o sistema informa que não há recomendações sociais disponíveis.
Pós-condições	O usuário visualiza recomendações baseadas em informações públicas de seus amigos.
Regras de Negócio	O sistema deve considerar apenas informações públicas ou autorizadas dos amigos adicionados.

A tabela a seguir apresenta a especificação do caso de uso UC-18 - Visualizar Perfil, responsável por permitir a consulta de informações do próprio perfil ou de perfis públicos de amigos.

Tabela 20 – Caso de Uso UC-18 - Visualizar Perfil

Objetivo	Permitir que o usuário visualize informações de seu perfil ou de perfis públicos de amigos.
RF/RNF Relacionados	RF17, RF20, RNF01
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve estar autenticado.
Frequência de uso	Média
Criticalidade	Baixa
Condição de Entrada	Usuário acessa a tela de perfil.
Fluxo Principal	1. O usuário acessa a área de perfil. 2. O sistema recupera as informações do perfil solicitado. 3. O sistema organiza os dados disponíveis para exibição. 4. O sistema apresenta o perfil ao usuário.
Fluxo Alternativo	Caso o perfil solicitado não exista ou não esteja disponível, o sistema exibe uma mensagem informando que o perfil não pôde ser encontrado.
Pós-condições	As informações do perfil são exibidas ao usuário.
Regras de Negócio	Informações privadas de outros usuários não devem ser exibidas sem autorização.

A tabela a seguir apresenta a especificação do caso de uso UC-19 - Editar Perfil, relacionado à alteração das informações do perfil do usuário autenticado.

Tabela 21 – Caso de Uso UC-19 - Editar Perfil

Objetivo	Permitir que o usuário altere suas informações de perfil.
RF/RNF Relacionados	RF20, RNF06, RNF08
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve estar autenticado.
Frequência de uso	Baixa
Criticalidade	Média
Condição de Entrada	Usuário seleciona a opção de editar perfil.

Fluxo Principal	1. O usuário acessa seu perfil. 2. O usuário seleciona a opção de edição. 3. O sistema exibe os dados editáveis. 4. O usuário altera as informações desejadas. 5. O sistema valida os dados informados. 6. O sistema salva as alterações no perfil.
Fluxo Alternativo	Caso os dados informados sejam inválidos, o sistema exibe uma mensagem de erro e mantém as informações anteriores.
Pós-condições	As informações do perfil do usuário são atualizadas.
Regras de Negócio	O usuário só pode editar o próprio perfil.

A tabela a seguir apresenta a especificação do caso de uso UC-20 - Adicionar Amigo, responsável por permitir a criação de vínculos entre usuários da plataforma.

Tabela 22 – Caso de Uso UC-20 - Adicionar Amigo

Objetivo	Permitir que o usuário adicione outros usuários à sua lista de amigos.
RF/RNF Relacionados	RF16, RNF05, RNF06
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve estar autenticado e o usuário a ser adicionado deve possuir uma conta cadastrada.
Frequência de uso	Média
Criticalidade	Média
Condição de Entrada	Usuário seleciona a opção de adicionar amigo.
Fluxo Principal	1. O usuário acessa a área social do sistema. 2. O usuário pesquisa por outro usuário. 3. O sistema exibe os usuários correspondentes à busca. 4. O usuário seleciona a opção de adicionar amigo. 5. O sistema registra a solicitação ou adiciona o usuário à lista de amigos.
Fluxo Alternativo	Caso o usuário pesquisado não seja encontrado, o sistema exibe uma mensagem informando que não há resultados para a busca.
Pós-condições	O usuário selecionado passa a fazer parte da lista de amigos ou recebe uma solicitação de amizade.
Regras de Negócio	Um usuário não pode adicionar a si mesmo como amigo.

A tabela a seguir apresenta a especificação do caso de uso UC-21 - Remover Amigo, relacionado à remoção de usuários da lista de amigos.

Tabela 23 – Caso de Uso UC-21 - Remover Amigo

Objetivo	Permitir que o usuário remova outro usuário de sua lista de amigos.
RF/RNF Relacionados	RF16, RNF06
Atores	Usuário
Prioridade	Baixa
Pré-condições	O usuário deve estar autenticado e possuir ao menos um amigo adicionado.
Frequência de uso	Baixa
Criticalidade	Baixa
Condição de Entrada	Usuário seleciona a opção de remover amigo.

Fluxo Principal	1. O usuário acessa sua lista de amigos. 2. O usuário seleciona um amigo cadastrado. 3. O usuário escolhe a opção de remoção. 4. O sistema solicita confirmação da ação. 5. O usuário confirma a remoção. 6. O sistema remove o amigo da lista.
Fluxo Alternativo	Caso o usuário cancele a confirmação, nenhuma alteração é realizada.
Pós-condições	O usuário selecionado deixa de fazer parte da lista de amigos.
Regras de Negócio	A remoção de amizade deve afetar apenas o vínculo entre os usuários, sem excluir nenhuma conta.

A tabela a seguir apresenta a especificação do caso de uso UC-22 - Visualizar Atividade de Amigos, responsável por permitir a consulta de atividades públicas de usuários adicionados como amigos.

Tabela 24 – Caso de Uso UC-22 - Visualizar Atividade de Amigos

Objetivo	Permitir que o usuário visualize atividades públicas de seus amigos.
RF/RNF Relacionados	RF17, RNF01, RNF07
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve estar autenticado e possuir amigos adicionados.
Frequência de uso	Média
Criticalidade	Baixa
Condição de Entrada	Usuário acessa a área de atividades dos amigos.
Fluxo Principal	1. O usuário acessa a área social do sistema. 2. O usuário seleciona a opção de visualizar atividades de amigos. 3. O sistema recupera as atividades públicas dos amigos adicionados. 4. O sistema organiza as atividades por relevância ou data. 5. O sistema exibe as atividades ao usuário.
Fluxo Alternativo	Caso não existam atividades disponíveis, o sistema exibe uma mensagem informando que não há atividades recentes.
Pós-condições	O usuário visualiza as atividades públicas de seus amigos.
Regras de Negócio	O sistema deve exibir apenas informações públicas ou autorizadas pelos usuários.

A tabela a seguir apresenta a especificação do caso de uso UC-23 - Visualizar Biblioteca de Amigos, relacionado à visualização dos jogos públicos cadastrados na biblioteca de amigos.

Tabela 25 – Caso de Uso UC-23 - Visualizar Biblioteca de Amigos

Objetivo	Permitir que o usuário visualize jogos públicos cadastrados na biblioteca de seus amigos.
RF/RNF Relacionados	RF17, RNF01, RNF07
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve estar autenticado e possuir amigos adicionados.
Frequência de uso	Média
Criticalidade	Baixa
Condição de Entrada	Usuário acessa o perfil ou a biblioteca de um amigo.
Fluxo Principal	1. O usuário acessa sua lista de amigos. 2. O usuário seleciona um amigo. 3. O usuário escolhe a opção de visualizar biblioteca. 4. O sistema recupera os jogos públicos da biblioteca do amigo. 5. O sistema exibe a biblioteca do amigo ao usuário.

Fluxo Alternativo	Caso a biblioteca do amigo esteja vazia ou privada, o sistema exibe uma mensagem informando que não há jogos disponíveis para visualização.
Pós-condições	O usuário visualiza os jogos públicos da biblioteca do amigo selecionado.
Regras de Negócio	O sistema deve respeitar as configurações de privacidade do usuário dono da biblioteca.

A tabela a seguir apresenta a especificação do caso de uso UC-24 - Visualizar *Dashboard*, responsável por apresentar informações analíticas sobre o perfil de utilização do usuário.

Tabela 26 – Caso de Uso UC-24 - Visualizar *Dashboard*

Objetivo	Permitir que o usuário visualize informações analíticas sobre sua utilização da plataforma.
RF/RNF Relacionados	RF14, RNF01, RNF07
Atores	Usuário
Prioridade	Média
Pré-condições	O usuário deve estar autenticado e possuir dados de utilização registrados.
Frequência de uso	Média
Criticalidade	Média
Condição de Entrada	Usuário acessa a área de <i>dashboard</i> .
Fluxo Principal	1. O usuário acessa a tela de <i>dashboard</i>. 2. O sistema recupera os dados de utilização do usuário. 3. O sistema calcula informações analíticas, como tempo de jogo, jogos mais utilizados e preferências do usuário. 4. O sistema organiza os dados em gráficos, indicadores ou listas. 5. O sistema apresenta o <i>dashboard</i> ao usuário.
Fluxo Alternativo	Caso não existam dados suficientes, o sistema exibe o <i>dashboard</i> vazio ou uma mensagem informando que ainda não há informações disponíveis.
Pós-condições	O usuário visualiza informações analíticas sobre seu perfil de utilização.
Regras de Negócio	O <i>dashboard</i> deve ser gerado com base apenas nos dados associados ao usuário autenticado.

APÊNDICE B – Protótipo

2. Protótipo

Este apêndice apresenta as telas do protótipo atual do *Salsilauncher*. As figuras demonstram a organização visual planejada para a aplicação, incluindo autenticação de usuários, navegação principal, biblioteca de jogos e funcionalidades de cadastro.

A Figura 4 apresenta a tela de login da aplicação. Nessa tela, o usuário informa e-mail e senha para acessar sua conta no sistema.

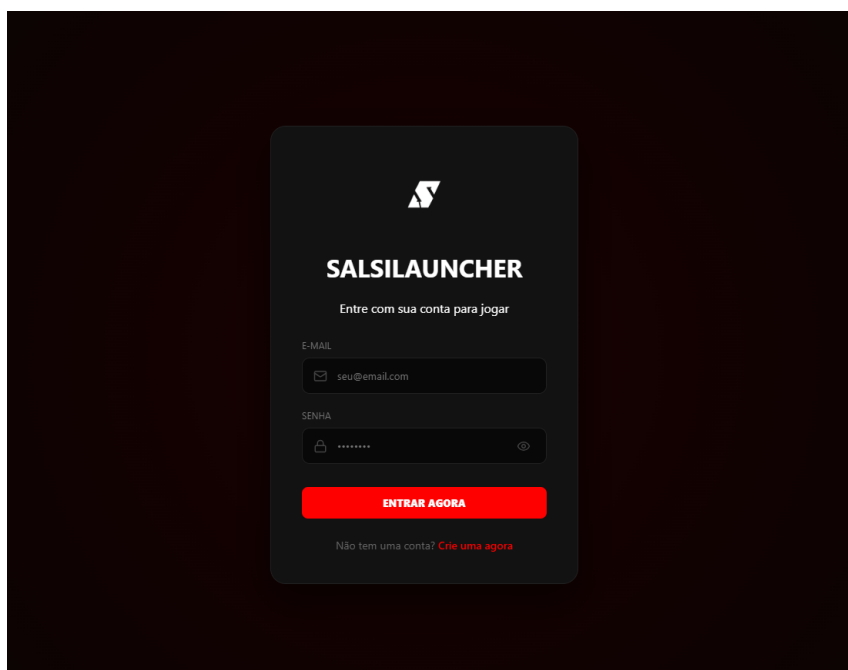


Figura 4 – Protótipo da tela de login

A Figura 5 apresenta a tela de criação de conta. Nela, o usuário informa nome, e-mail e senha para realizar seu cadastro na plataforma.

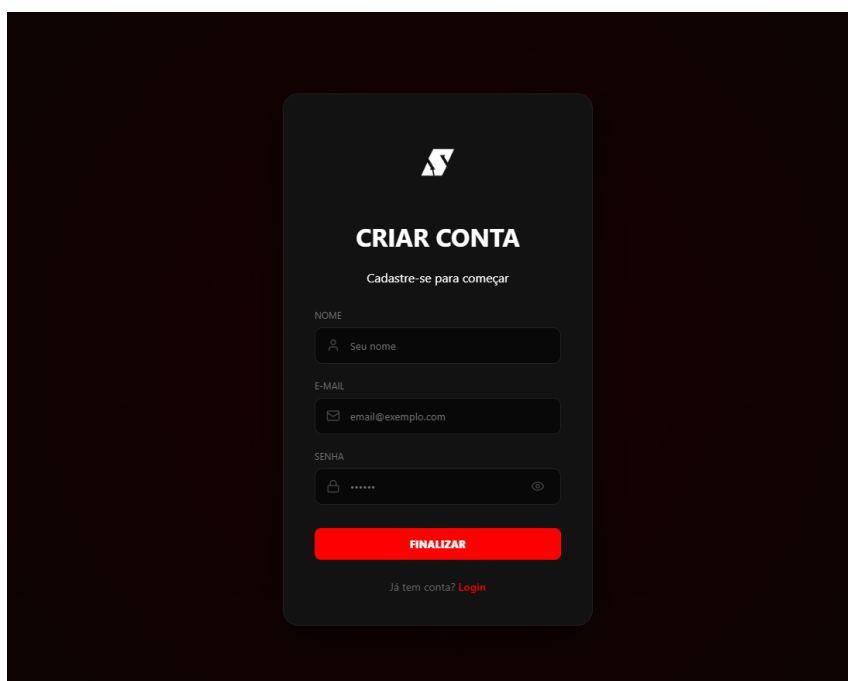


Figura 5 – Protótipo da tela de criação de conta

A Figura 6 apresenta a tela inicial da aplicação após o login. Essa tela exibe a área principal da biblioteca, a barra de pesquisa, o menu lateral e a identificação do usuário autenticado.

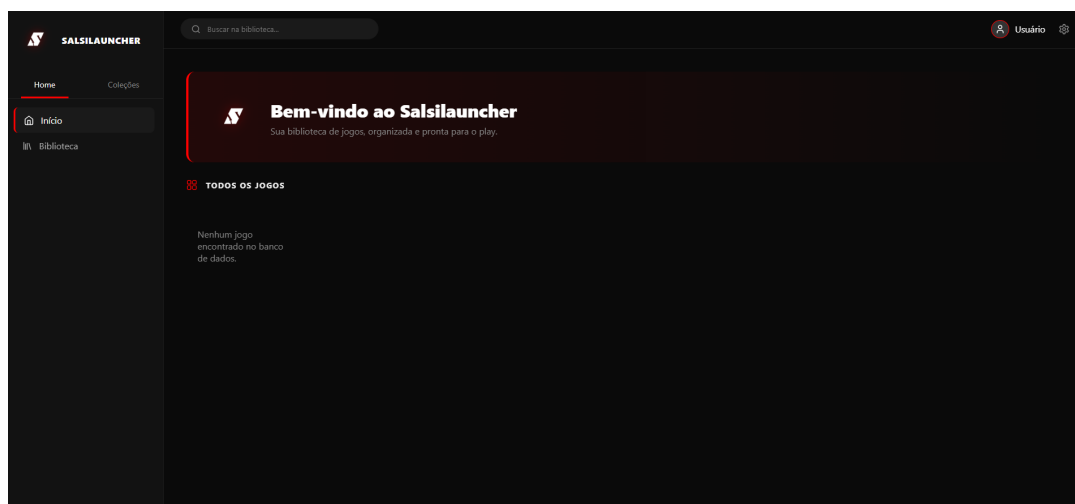


Figura 6 – Protótipo da tela inicial da biblioteca

A Figura 7 apresenta o menu lateral com a aba inicial selecionada. Esse componente concentra os principais caminhos de navegação relacionados à biblioteca do usuário.

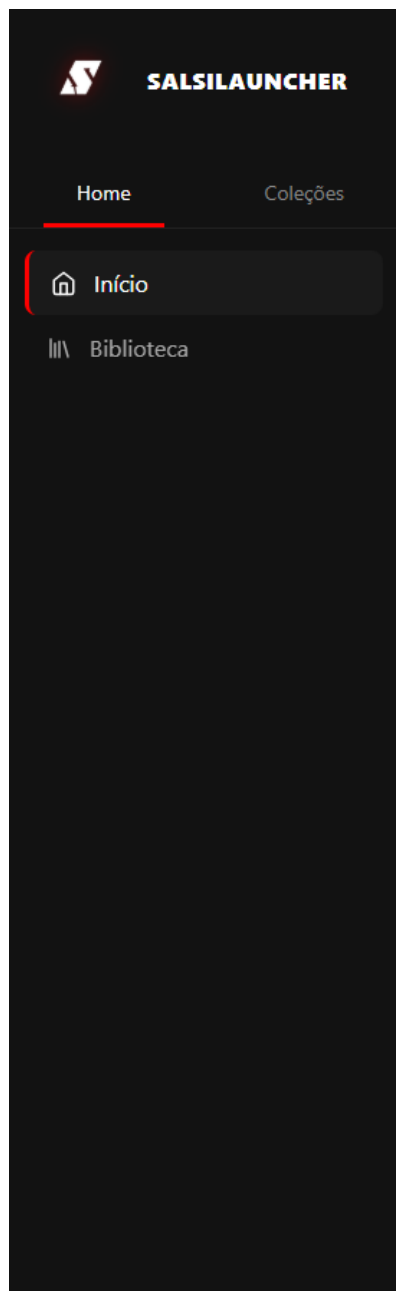


Figura 7 – Protótipo do menu lateral na aba inicial

A Figura 8 apresenta o menu lateral com a aba de coleções selecionada. Essa área permite acessar recursos como favoritos, criação de novo jogo e escaneamento de pastas.

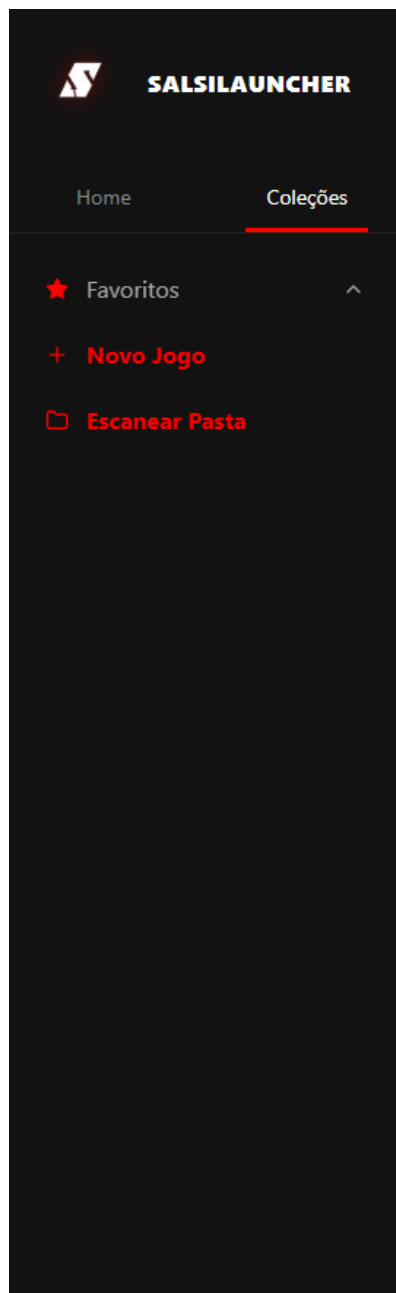
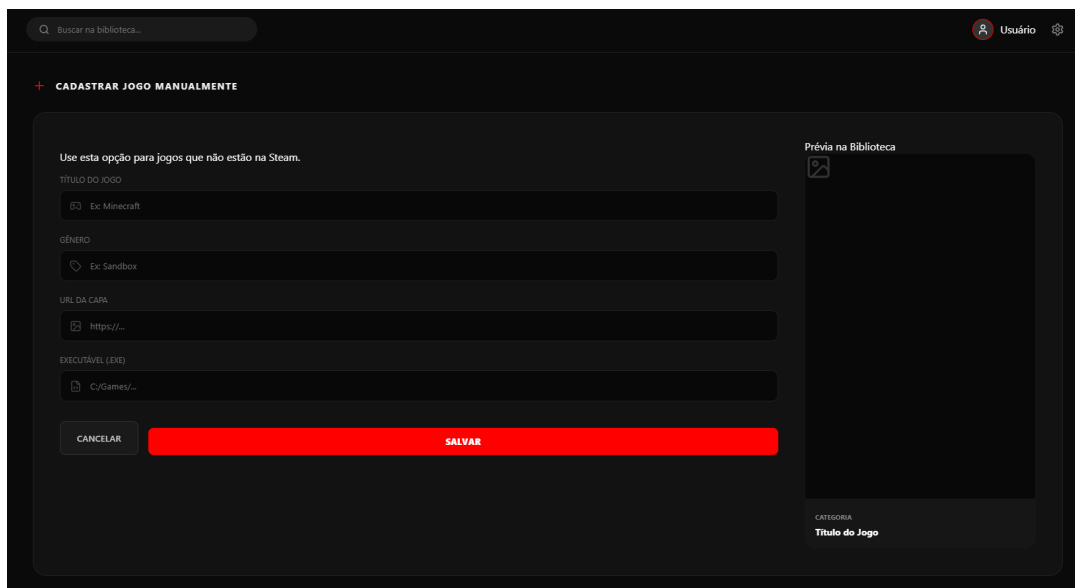


Figura 8 – Protótipo do menu lateral na aba de coleções

A Figura 9 apresenta a tela de cadastro manual de jogos. Nessa tela, o usuário pode informar dados como título, gênero, URL da capa e caminho do executável, além de visualizar uma prévia do jogo antes de salvá-lo.



Q Buscar na biblioteca...

Usuário

+ CADASTRAR JOGO MANUALMENTE

Use esta opção para jogos que não estão na Steam.

TÍTULO DO JOGO

Ex: Minecraft

GÊNERO

Ex: Sandbox

URL DA CAPA

https://...

EXECUTÁVEL (.EXE)

C:/Games/...

CANCELAR

SALVAR

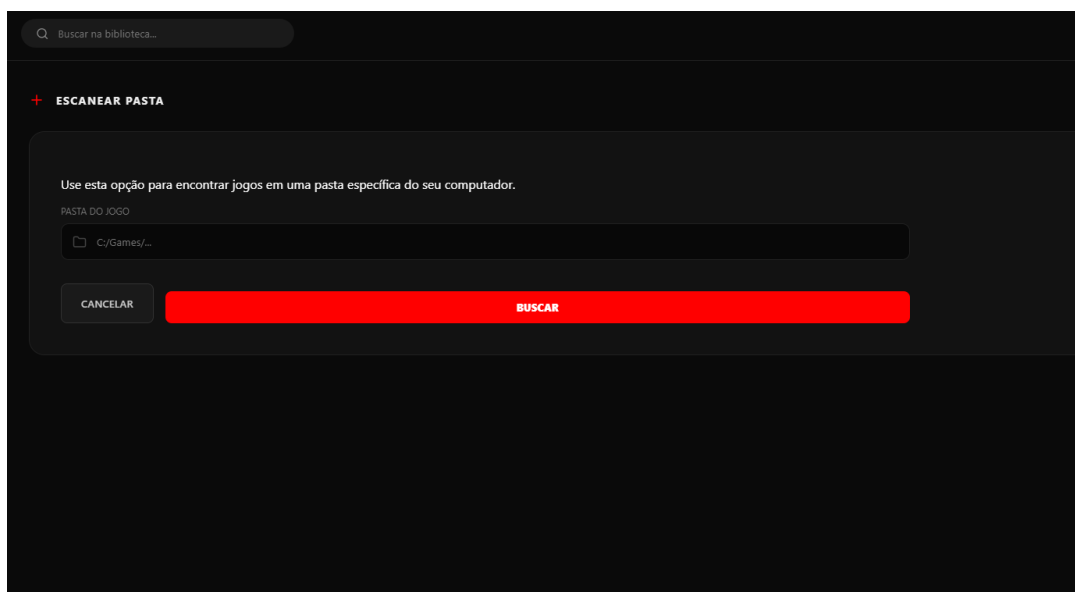
Prévia na Biblioteca

CATEGORIA

Título do Jogo

Figura 9 – Protótipo da tela de cadastro manual de jogo

A Figura 10 apresenta a tela de escaneamento de pasta. Essa funcionalidade permite que o usuário informe um diretório do computador para que o sistema busque jogos ou executáveis presentes naquela pasta.



Q Buscar na biblioteca...

+ ESCANEAR PASTA

Use esta opção para encontrar jogos em uma pasta específica do seu computador.

PASTA DO JOGO

C:/Games/...

CANCELAR

BUSCAR

Figura 10 – Protótipo da tela de escaneamento de pasta

APÊNDICE C – Tabelas de Casos de Teste

3. Tabelas de Casos de Teste

Este apêndice apresenta os casos de teste planejados para validar as funcionalidades do *Salsilauncher*. Os testes foram organizados conforme os módulos do sistema, contemplando biblioteca, execução e monitoramento, integrações externas, sistema de recomendação, funcionalidades sociais, *dashboard* analítico, coleções e requisitos funcionais e não funcionais.

3.1. Gerenciamento da Biblioteca

A tabela a seguir apresenta o caso de teste CT-01 - Cadastro de jogo, elaborado para validar se um jogo pode ser adicionado corretamente à biblioteca do usuário.

Tabela 27 – Caso de teste CT-01

Identificador	CT-01
Funcionalidade	Cadastro de jogo
Objetivo	Verificar se um jogo pode ser adicionado à biblioteca.
Pré-condição	Nenhuma.
Entrada	Nome do jogo e caminho válido do executável.
Resultado esperado	O jogo deve ser salvo e exibido na biblioteca do usuário.
Exceção (Caminho Triste)	Se o caminho do executável for inválido ou inexistente, o sistema deve barrar o cadastro e exibir uma mensagem de erro ("Caminho do arquivo não encontrado").

A tabela a seguir apresenta o caso de teste CT-02 - Edição de jogo, elaborado para verificar se as informações de um jogo cadastrado podem ser alteradas corretamente.

Tabela 28 – Caso de teste CT-02

Identificador	CT-02
Funcionalidade	Edição de jogo
Objetivo	Verificar a alteração dos dados cadastrados de um jogo.
Pré-condição	Jogo já cadastrado e visível na biblioteca.
Entrada	Alteração de informações previamente cadastradas (ex: nome ou imagem da capa).
Resultado esperado	Os dados devem ser atualizados corretamente no banco e na interface.
Exceção (Caminho Triste)	Se o formulário for submetido com campos obrigatórios em branco, a edição não deve ser concluída.

A tabela a seguir apresenta o caso de teste CT-03 - Remoção de jogo, elaborado para validar a exclusão de um jogo da biblioteca do usuário.

Tabela 29 – Caso de teste CT-03

Identificador	CT-03
Funcionalidade	Remoção de jogo
Objetivo	Verificar a exclusão de um jogo da biblioteca.
Pré-condição	Jogo cadastrado no sistema.
Entrada	Seleção de um jogo existente e confirmação de exclusão.
Resultado esperado	O jogo deve ser removido do banco de dados e deixar de aparecer na biblioteca.
Exceção (Caminho Triste)	N/A

A tabela a seguir apresenta o caso de teste CT-04 - Pesquisa de jogos, elaborado para verificar se a busca exibe corretamente os jogos compatíveis com o termo informado.

Tabela 30 – Caso de teste CT-04

Identificador	CT-04
Funcionalidade	Pesquisa de jogos
Objetivo	Verificar a busca por jogos cadastrados.
Pré-condição	Biblioteca contendo um ou mais jogos cadastrados.
Entrada	Nome completo ou parcial de um jogo inserido na barra de pesquisa.
Resultado esperado	O sistema deve filtrar e exibir apenas os jogos compatíveis com a pesquisa.
Exceção (Caminho Triste)	Se o termo pesquisado não corresponder a nenhum jogo, exibir mensagem "Nenhum jogo encontrado".

3.2. Execução e Monitoramento

A tabela a seguir apresenta o caso de teste CT-05 - Execução de jogo, elaborado para validar a inicialização do executável associado a um jogo cadastrado.

Tabela 31 – Caso de teste CT-05

Identificador	CT-05
Funcionalidade	Execução de jogo
Objetivo	Verificar a inicialização do executável associado ao jogo.
Pré-condição	Caminho do executável íntegro e arquivo existente no disco.
Entrada	Comando de execução (clique em "Jogar").
Resultado esperado	O processo do jogo deve ser iniciado corretamente no sistema operacional.
Exceção (Caminho Triste)	Se o executável tiver sido movido, exibir erro alertando que o jogo não pode ser executado.

A tabela a seguir apresenta o caso de teste CT-06 - Registro de tempo de uso, elaborado para verificar se o sistema registra corretamente a duração da sessão de jogo.

Tabela 32 – Caso de teste CT-06

Identificador	CT-06
Funcionalidade	Registro de tempo de uso
Objetivo	Verificar o monitoramento da duração da sessão de jogo.
Pré-condição	Execução do jogo através do CT-05 realizada com sucesso.
Entrada	Acompanhamento do processo e encerramento do jogo.
Resultado esperado	O tempo total em minutos ou horas deve ser somado e salvo no perfil do usuário.
Exceção (Caminho Triste)	Se o <i>launcher</i> for fechado de forma forçada antes do jogo, o tempo parcial registrado até a falha deve ser persistido, se possível.

A tabela a seguir apresenta o caso de teste CT-07 - Prevenção de dupla execução, elaborado para validar se o sistema impede a abertura de múltiplas instâncias do mesmo jogo.

Tabela 33 – Caso de teste CT-07

Identificador	CT-07
Funcionalidade	Prevenção de dupla execução
Objetivo	Verificar se o sistema impede a inicialização de uma nova instância de um jogo já aberto.
Pré-condição	O jogo já foi iniciado e seu processo está ativo no sistema operacional.
Entrada	Tentativa de executar novamente o mesmo jogo.
Resultado esperado	O sistema deve bloquear a ação, não iniciar um novo processo e exibir uma notificação/alerta informando: "Este jogo já está em execução".
Exceção (Caminho Triste)	Se o <i>launcher</i> perder o rastreamento do processo e permitir a execução dupla, o teste deve falhar.

3.3. Integração com APIs Externas

A tabela a seguir apresenta o caso de teste CT-08 - Importação de metadados, elaborado para verificar se o sistema obtém automaticamente informações de jogos a partir de fontes externas.

Tabela 34 – Caso de teste CT-08

Identificador	CT-08
Funcionalidade	Importação de metadados
Objetivo	Verificar a obtenção automática de informações do jogo.
Pré-condição	Conexão com a internet ativa e API de metadados operante.
Entrada	Captura dos metadados de um jogo com título presente na base de dados externa.
Resultado esperado	Nome, descrição, capa e categorias devem ser preenchidos automaticamente.
Exceção (Caminho Triste)	Se a API estiver indisponível ou retornar timeout, permitir inserção manual dos metadados sem travar o <i>launcher</i> .

A tabela a seguir apresenta o caso de teste CT-09 - Sincronização da biblioteca *Steam*, elaborado para validar a importação de jogos a partir de uma conta *Steam* vinculada.

Tabela 35 – Caso de teste CT-09

Identificador	CT-09
Funcionalidade	Sincronização da biblioteca <i>Steam</i>
Objetivo	Verificar a importação de jogos de uma conta vinculada.
Pré-condição	Perfil <i>Steam</i> configurado como público.
Entrada	SteamID válido informado pelo usuário.
Resultado esperado	Os jogos da biblioteca <i>Steam</i> informada devem ser importados para o <i>Salsilauncher</i> .
Exceção (Caminho Triste)	Se o SteamID for inválido ou privado, exibir mensagem informando que não foi possível realizar a leitura dos jogos.

4. Sistema de Recomendação

A tabela a seguir apresenta o caso de teste CT-10 - Geração de recomendações, elaborado para verificar se o sistema apresenta sugestões compatíveis com o perfil do usuário.

Tabela 36 – Caso de teste CT-10

Identificador	CT-10
Funcionalidade	Geração de recomendações
Objetivo	Verificar a geração de sugestões com base no perfil do usuário.
Pré-condição	Usuário com histórico de horas jogadas e categorias definidas no banco de dados.
Entrada	Acesso à aba de recomendações.
Resultado esperado	O sistema deve apresentar uma lista de sugestões compatíveis com o perfil do usuário.
Exceção (Caminho Triste)	Se o não possuir dados, exibir uma lista genérica de jogos populares.

4.1. Sistema Social

A tabela a seguir apresenta o caso de teste CT-11 - Adição de amigos, elaborado para validar a criação de vínculos entre usuários cadastrados na plataforma.

Tabela 37 – Caso de teste CT-11

Identificador	CT-11
Funcionalidade	Adição de amigos
Objetivo	Verificar a criação de vínculos entre usuários.
Pré-condição	Ambos os usuários estão cadastrados na plataforma.
Entrada	Envio de solicitação e respectivo aceite.
Resultado esperado	Os usuários devem aparecer mutuamente em suas listas de amigos.
Exceção (Caminho Triste)	Se o usuário tentar adicionar um ID inexistente, exibir "Usuário não encontrado".

A tabela a seguir apresenta o caso de teste CT-12 - Visualização de atividades, elaborado para verificar a exibição de atividades públicas de amigos.

Tabela 38 – Caso de teste CT-12

Identificador	CT-12
Funcionalidade	Visualização de atividades
Objetivo	Verificar a exibição das atividades em tempo real de amigos.
Pré-condição	Usuário possui amigos cadastrados com atividades recentes.
Entrada	Navegação para a aba social.
Resultado esperado	O sistema deve exibir informações públicas como "Jogando agora" ou "Último jogo acessado".
Exceção (Caminho Triste)	Se o amigo alterar a privacidade para "Invisível", sua atividade não deve ser exibida.

4.2. Dashboard Analítico

A tabela a seguir apresenta o caso de teste CT-13 - *Dashboard* analítico, elaborado para validar a geração e apresentação de estatísticas relacionadas ao uso da plataforma.

Tabela 39 – Caso de teste CT-13

Identificador	CT-13
Funcionalidade	<i>Dashboard</i> analítico
Objetivo	Verificar a geração das estatísticas e métricas do usuário.
Pré-condição	Possuir registros válidos de tempo de uso associados à conta.
Entrada	Acesso ao menu de estatísticas/ <i>dashboard</i> .
Resultado esperado	O sistema deve apresentar gráficos e indicadores corretamente renderizados.
Exceção (Caminho Triste)	Caso o cálculo estatístico seja custoso, a tela deve exibir um <i>loader</i> até a conclusão dos dados.

4.3. Gerenciamento de Pastas e Coleções

A tabela a seguir apresenta o caso de teste CT-14 - Escanear pasta, elaborado para verificar o mapeamento automático de jogos a partir de um diretório selecionado.

Tabela 40 – Caso de teste CT-14

Identificador	CT-14
Funcionalidade	Escanear pasta
Objetivo	Verificar o mapeamento automático e adição em massa de jogos a partir de um diretório.
Pré-condição	Pasta contendo um ou mais diretórios ou executáveis de jogos.
Entrada	Caminho da pasta selecionado pelo usuário.
Resultado esperado	O sistema deve identificar jogos, listá-los para confirmação e adicioná-los à biblioteca.
Exceção (Caminho Triste)	Se a pasta estiver vazia ou sem permissão de leitura, exibir mensagem de erro apropriada.

A tabela a seguir apresenta o caso de teste CT-15 - Criar coleção, elaborado para validar a criação de coleções personalizadas de jogos.

Tabela 41 – Caso de teste CT-15

Identificador	CT-15
Funcionalidade	Criar coleção
Objetivo	Verificar a capacidade de criar grupos personalizados para categorizar jogos.
Pré-condição	Nenhuma.
Entrada	Inserção de um nome válido para a nova coleção.
Resultado esperado	A coleção deve ser criada, persistida e exibida na interface.
Exceção (Caminho Triste)	Não deve ser permitido salvar uma coleção com nome em branco.

A tabela a seguir apresenta o caso de teste CT-16 - Editar coleção, elaborado para verificar a alteração das propriedades e do conteúdo de uma coleção existente.

Tabela 42 – Caso de teste CT-16

Identificador	CT-16
Funcionalidade	Editar coleção
Objetivo	Verificar a alteração das propriedades e do conteúdo de uma coleção.
Pré-condição	Uma coleção já existente.
Entrada	Novo nome e/ou alteração dos jogos associados.
Resultado esperado	O nome e os jogos vinculados devem refletir as alterações realizadas.
Exceção (Caminho Triste)	Não deve ser permitido salvar uma coleção com nome em branco.

A tabela a seguir apresenta o caso de teste CT-17 - Remover coleção, elaborado para validar a exclusão de uma coleção sem remover os jogos associados a ela.

Tabela 43 – Caso de teste CT-17

Identificador	CT-17
Funcionalidade	Remover coleção
Objetivo	Verificar a exclusão de uma coleção sem remover os jogos associados.
Pré-condição	Coleção existente selecionada na interface.
Entrada	Clique no botão de exclusão da coleção.
Resultado esperado	A coleção deve desaparecer da interface e os jogos devem permanecer na biblioteca.
Exceção (Caminho Triste)	N/A

A tabela a seguir apresenta o caso de teste CT-18 - Pesquisa de coleções, elaborado para verificar se a busca por coleções retorna apenas resultados compatíveis com o termo informado.

Tabela 44 – Caso de teste CT-18

Identificador	CT-18
Funcionalidade	Pesquisa de coleções
Objetivo	Verificar a funcionalidade de busca pelo nome das coleções.
Pré-condição	Múltiplas coleções cadastradas.
Entrada	Digitação completa ou parcial do nome da coleção.
Resultado esperado	A interface deve exibir apenas as coleções compatíveis com a pesquisa.
Exceção (Caminho Triste)	Se não houver correspondência, exibir "Nenhuma coleção encontrada".

4.4. Validação de Requisitos Não Funcionais

A tabela a seguir apresenta o caso de teste CT-19 - Performance de Consulta no *Backend*, elaborado para validar se operações de consulta mais pesadas respondem dentro de um tempo aceitável.

Tabela 45 – Caso de teste CT-19

Identificador	CT-19
Funcionalidade	Performance de Consulta no <i>Backend</i>
Objetivo	Garantir que <i>endpoints</i> pesados respondam dentro de limites aceitáveis.
Pré-condição	Banco de dados populado com uma carga média realista de dados.
Entrada	Requisição GET para geração de estatísticas.
Resultado esperado	A API deve processar e devolver a resposta em tempo aceitável (inferior a 1,5 segundos).
Exceção (Caminho Triste)	Caso a resposta ultrapasse o tempo estipulado, o teste automatizado deve falhar, indicando necessidade de otimização.